

DUOC UC

Taller Aplicado de Programación (TPY1101)

**INFORME TÉCNICO
ESTADO DE AVANCE N° 2**

MaduraApp

Sistema de Análisis de Madurez Agrícola mediante Visión Computacional

Integrantes

Claudio Vicente Aro Kath · RUT: 22.022.498-8

Docente: José Ignacio Campos Árevalo

Sección: 001D

Mayo 2026

1. Introducción

El presente informe documenta el Estado de Avance 2 del proyecto MaduraApp, sistema de análisis de madurez agrícola mediante visión computacional. El documento se construye sobre los entregables de la Evaluación 1 y avanza hacia el diseño arquitectónico formal, la configuración de ambientes y la demostración del producto funcional.

1.1 Propósito

Este informe responde a los Indicadores de Logro IL2.1, IL2.2 e IL2.3 de la asignatura TPY1101, estructurado en torno a tres ejes:

- Diseño metodológico riguroso — modelado arquitectónico bajo el estándar 4+1 de Philippe Kruchten.
- Configuración reproducible de ambientes — descripción detallada del servidor de producción y del ambiente de pruebas.
- Evidencia del producto funcional — desarrollo de calidad con buenas prácticas, pruebas automatizadas y procedimientos de gestión de datos.

1.2 Alcance

El informe cubre: documentación arquitectónica completa (5 vistas Kruchten), procedimientos operativos de despliegue y backup, plan de pruebas con cobertura operacional, validación y verificación, y evidencia de la suite automatizada (9/9 pytest backend, tests JVM Android).

2. Resumen Evaluación 1

Esta sección sintetiza los entregables aprobados en la Evaluación 1 y se incluye por requisito formal de continuidad del proyecto.

2.1 Problema identificado

Las pérdidas post-cosecha en frutas climatéricas (aguacate, plátano, tomate, mango) representan en Chile entre un 20% y un 40% de la producción según FAO/ODEPA. La causa raíz identificada en el diagrama de Ishikawa es la falta de criterios objetivos y accesibles para determinar el punto óptimo de consumo a nivel de consumidor final, pequeño productor y feria local.

2.2 Solución propuesta

MaduraApp es un sistema de visión computacional accesible desde dispositivos móviles que clasifica el estado de madurez de cuatro frutas climatéricas en tres categorías (Inmaduro / Óptimo / Sobre maduro) y entrega recomendaciones agronómicas contextualizadas en tiempo real.

2.3 Tecnologías seleccionadas

Componente	Tecnología	Versión
App móvil	Android nativo — Kotlin + CameraX + Room	API 29+
Backend	Python + FastAPI + SQLAlchemy async	3.12 / 0.135
Modelo IA	YOLO26n (Ultralytics)	Enero 2026
Base de datos	PostgreSQL (Supabase prod) / SQLite (dev)	16 / —
Contenerización	Docker + Docker Compose	—
Cloud	Render — plan Free, región Oregon	—
CI/CD	GitHub Actions	—
Control de versiones	Git + GitHub (2 repositorios)	—

2.4 Estado al cierre de Evaluación 1

- Sprint 1 Backend completo: API REST con 3 endpoints, persistencia async, migraciones Alembic, suite 9/9 pytest.
- Sprint 2 Android completo: app con CameraX, MVVM, cache offline con Room, tests JVM.
- Pipeline de entrenamiento CRISP-DM listo: 31.940 imágenes, 12 clases, 4 frutas.
- Modelo entrenado YOLO26n: mAP@50 = 0.9229 (KPI $\geq$ 0.75 superado con holgura).

3. Modelado y Diseño — Modelo 4+1 de Kruchten

El modelo de vistas arquitectónicas 4+1 propuesto por Philippe Kruchten (IEEE Software, noviembre 1995) describe la arquitectura de sistemas de software intensivos en computación mediante cinco vistas concurrentes, cada una orientada a un conjunto específico de stakeholders.

Vista	Audiencia principal	Preocupación
Lógica	Usuarios finales, analistas	Funcionalidad que provee el sistema
Procesos	Integradores de sistema	Performance, escalabilidad, concurrencia
Desarrollo	Programadores, gerentes	Organización del software, gestión
Física	Ingenieros de sistemas	Topología, comunicaciones, hardware
+1 Escenarios	Todos	Validación e integración de las 4 vistas

3.1 Vista Lógica

La Vista Lógica describe qué hace el sistema desde la perspectiva del usuario final. Se compone del diagrama de clases UML y un diagrama de componentes de alto nivel.

Entidades del dominio:

- FruitType — Enumerable: aguacate_hass, platano, tomate_usda, mango.
- MaturityLabel — Enumerable: INMADURO, OPTIMO, SOBRE_MADURO.
- ScanResult — Resultado de aplicar el modelo a una imagen: fruta, madurez, confianza, bbox, recomendación.
- ScanEntity — Persistencia de cada ScanResult vinculada a un token de usuario.

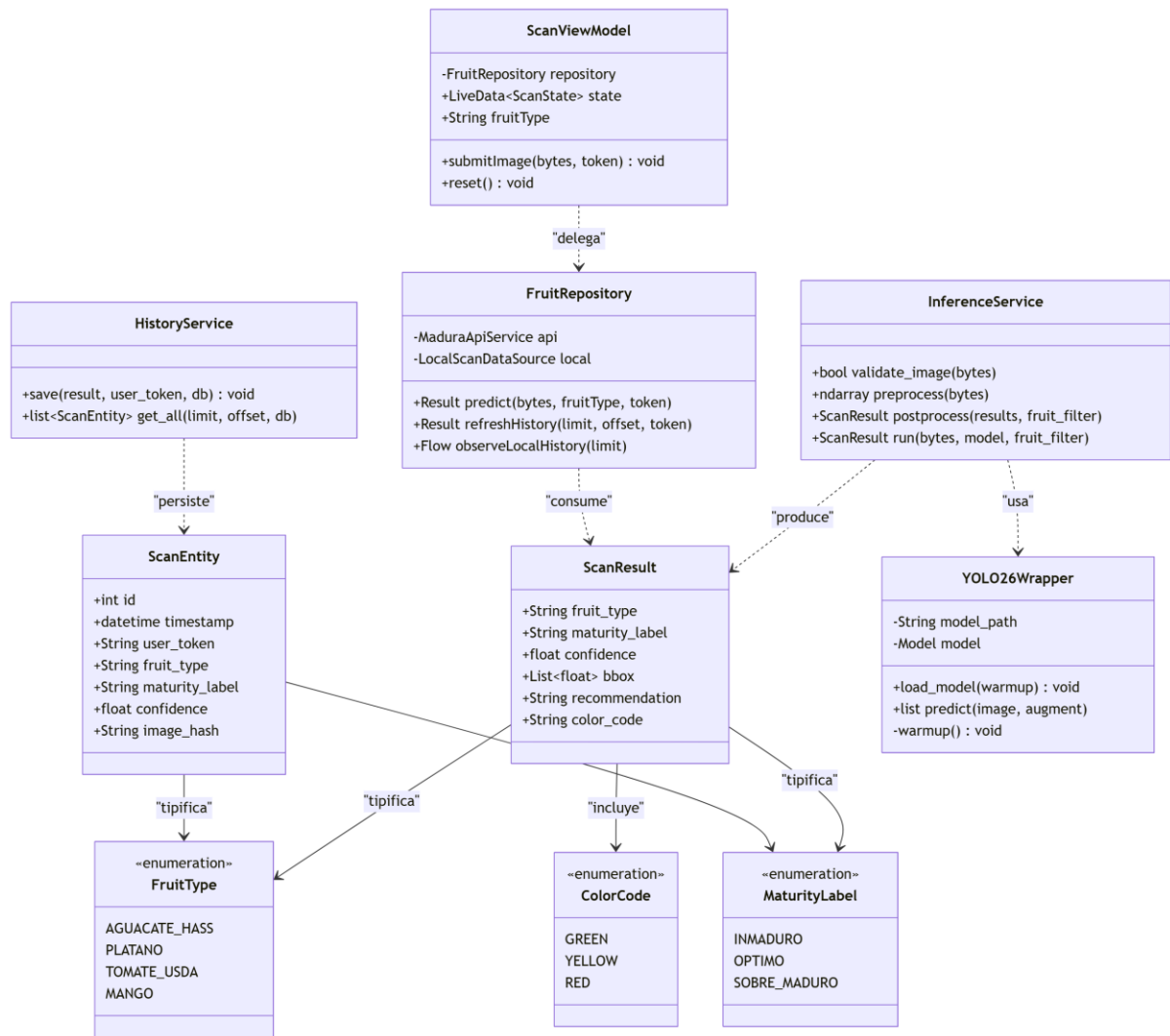


Figura 2. Diagrama de clases del dominio

Reglas de negocio:

- RN-01: Un escaneo es válido solo si la confianza supera el CONFIDENCE_THRESHOLD (0.55 configurable).
- RN-02: Con fruit_filter, el umbral se relaja al 50% del valor base (0.275) acotando clases.
- RN-03: Toda predicción exitosa se persiste en el historial.
- RN-04: Las imágenes nunca se persisten — solo el resultado (privacidad).

Componentes lógicos:

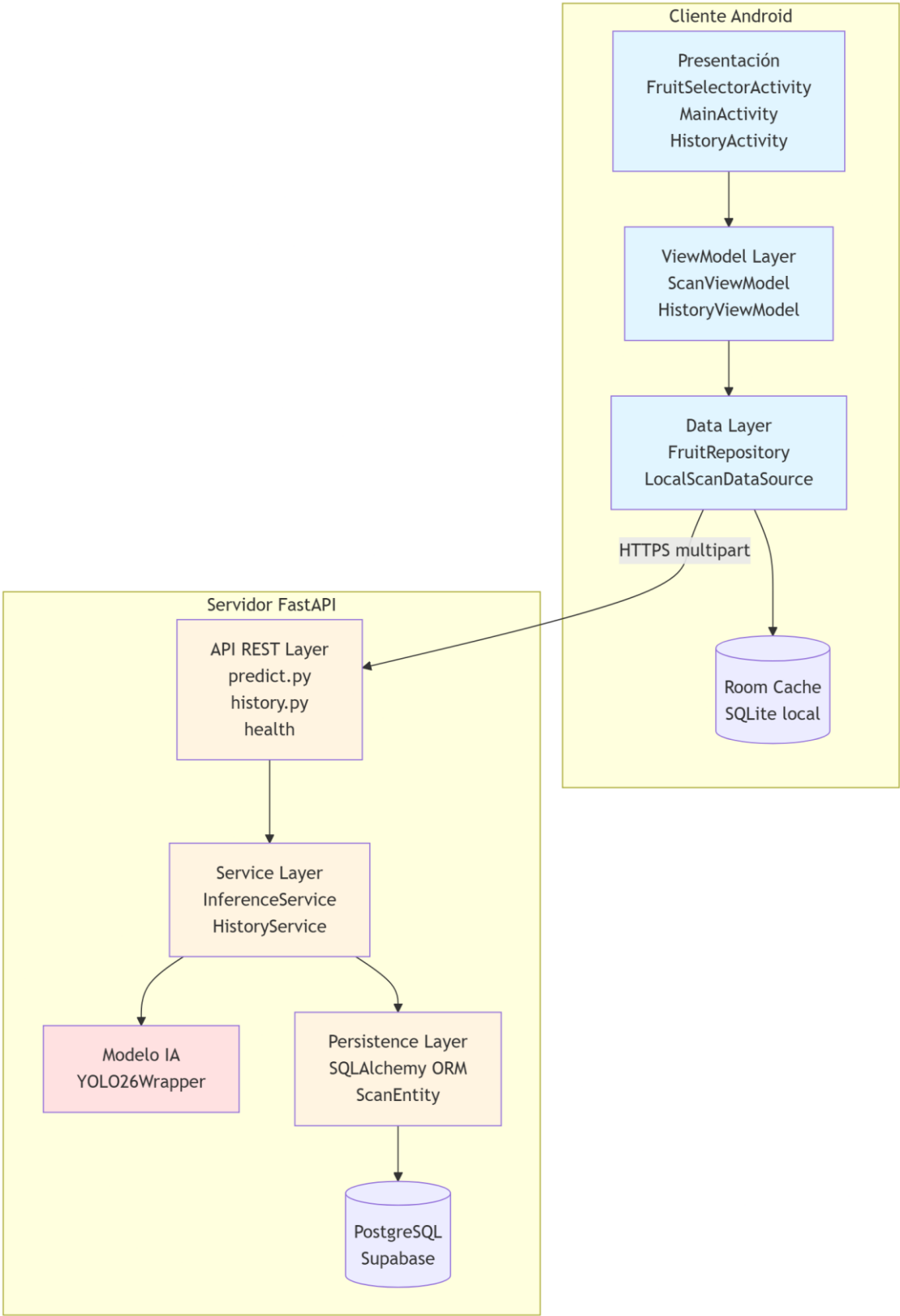


Figura 1. Diagrama de componentes lógicos del backend

Componente	Responsabilidad	Tecnología
Presentación móvil	Captura, navegación, render resultados	Activities + CameraX + ViewBinding

Componente	Responsabilidad	Tecnología
ViewModel	Estado de UI, llamadas al repositorio	AndroidX Lifecycle + LiveData + Coroutines
Data móvil	Orquestación API + cache local	Retrofit 2 + Room 2.6
API REST	Routing, validación de inputs	FastAPI 0.135
Servicios backend	Inferencia, persistencia, recomendaciones	Python 3.12
Modelo IA	Carga del modelo, predicción, TTA	Ultralytics + PyTorch
Persistence	Mapeo objeto-relacional, transacciones async	SQLAlchemy 2.0 + asyncpg

Trazabilidad con requisitos funcionales (ERS):

ID RF	Requerimiento	Componente que lo implementa
RF-01	Capturar imagen con cámara	MainActivity + CameraX
RF-02	Seleccionar imagen desde galería	MainActivity + ActivityResultContracts
RF-03	Pre-seleccionar fruta a escanear	FruitSelectorActivity
RF-04	Enviar imagen al backend	FruitRepository + MaduraApiService
RF-05	Clasificar madurez con IA	InferenceService + YOLO26Wrapper
RF-06	Devolver recomendación contextualizada	RECOMMENDATION_MAP en inference_service.py
RF-07	Persistir cada escaneo	HistoryService + ScanEntity
RF-08	Consultar historial paginado	history.py + HistoryService
RF-09	Visualizar historial offline	LocalScanDataSource + Room
RF-10	Semáforo visual de madurez	UI MainActivity + ColorCode
RF-11	Validar formato de imagen	predict.py (Content-Type allowlist)
RF-12	Health check del servicio	GET /v1/health

3.2 Vista de Procesos

La Vista de Procesos modela el comportamiento en tiempo de ejecución, abordando concurrencia, sincronización y performance.

Modelo de concurrencia:

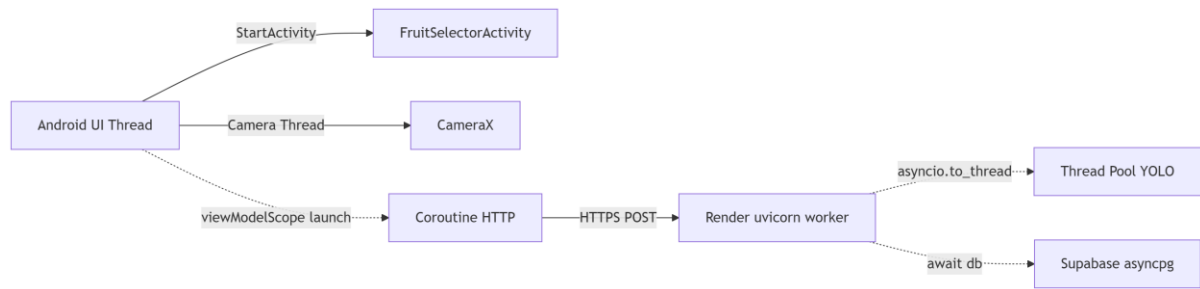


Figura 3. Modelo de concurrencia inter-sistemas

Sistema	Mecanismo	Propósito
Android	Coroutines Kotlin (viewModelScope)	HTTP sin bloquear hilo UI
Android	Hilo CameraX (SingleThreadExecutor)	Captura sin bloquear preview
FastAPI	Event loop asyncio	Atención de múltiples requests
FastAPI	asyncio.to_thread para YOLO	Inferencia CPU-bound sin bloquear event loop
FastAPI	AsyncSession SQLAlchemy	Queries BD sin bloquear

Flujo principal CU-01 Escaneo con cámara:

- 1. Usuario selecciona fruta en FruitSelectorActivity.
- 2. MainActivity lanza preview de cámara (CameraX).
- 3. Usuario toca Escanear → ScanState cambia a Loading.
- 4. FruitRepository.predict() arma MultipartBody (file + fruit_type).
- 5. POST HTTPS a /v1/predict en Render.
- 6. Backend valida inputs → asyncio.to_thread lanza inferencia YOLO con TTA.
- 7. InferenceService.postprocess() filtra por fruit_filter → ScanResult.
- 8. HistoryService.save() → INSERT en scans (Supabase).
- 9. Response → Android cachea en Room → ScanState = Success.
- 10. MainActivity renderiza semáforo + recomendación agronómica.

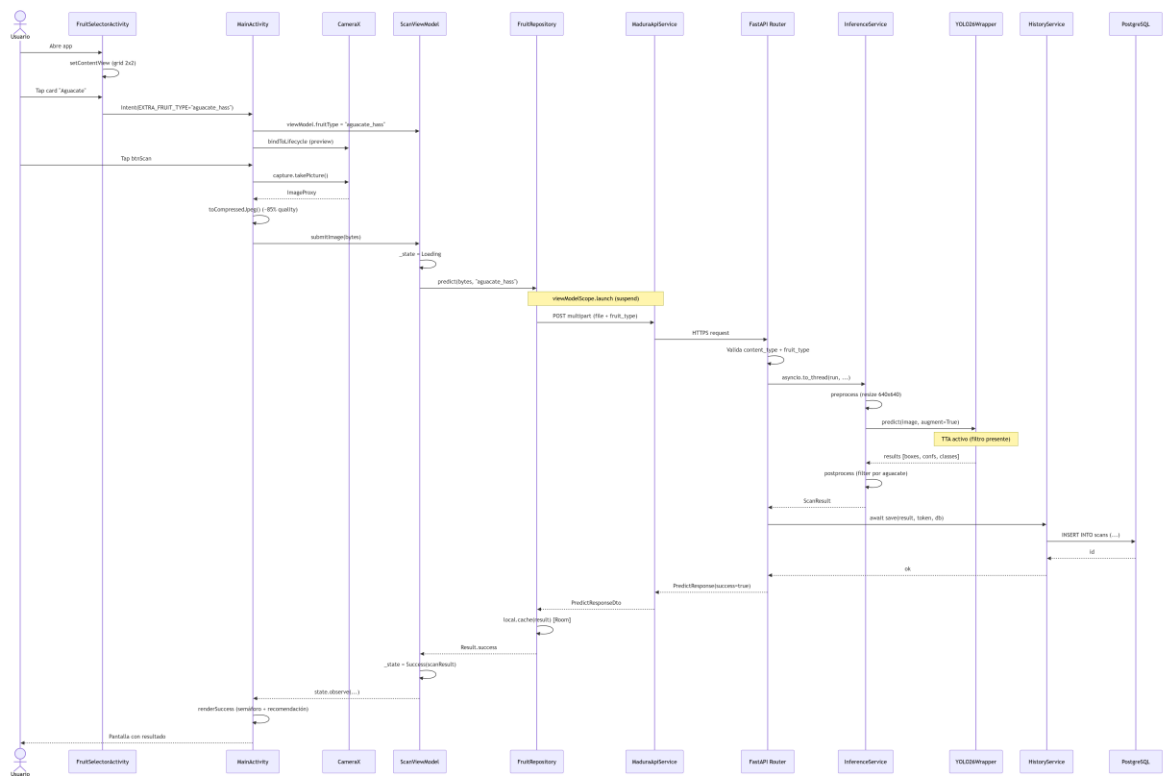


Figura 4. Diagrama de secuencia — CU-01 Escaneo con cámara

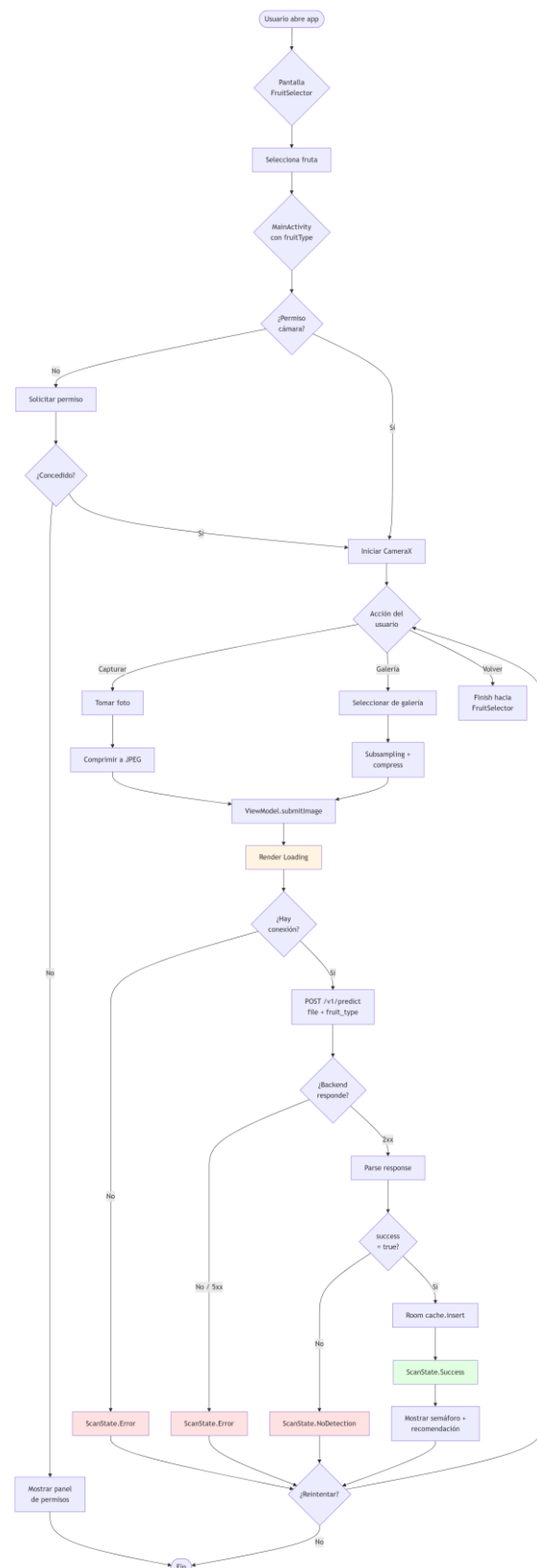


Figura 5. Diagrama de actividades — flujo de escaneo

Latencias esperadas:

Tramo	Tiempo CPU local	Tiempo Render Free
Captura + compresión JPEG	30–80 ms	50–150 ms
Inferencia YOLO sin TTA	80–150 ms	200–400 ms
Inferencia YOLO con TTA	160–300 ms	400–800 ms
INSERT en BD	5–10 ms	50–150 ms
Total user-perceived	~250–500 ms	~1.5–2.5 s

Máquina de estados ScanState:

- Idle → Loading: submitImage()
- Loading → Success: 2xx con data
- Loading → NoDetection: 2xx sin detección
- Loading → Error: falla de red o servidor
- Success / NoDetection / Error → Idle: reset() (botón Volver a escanear)

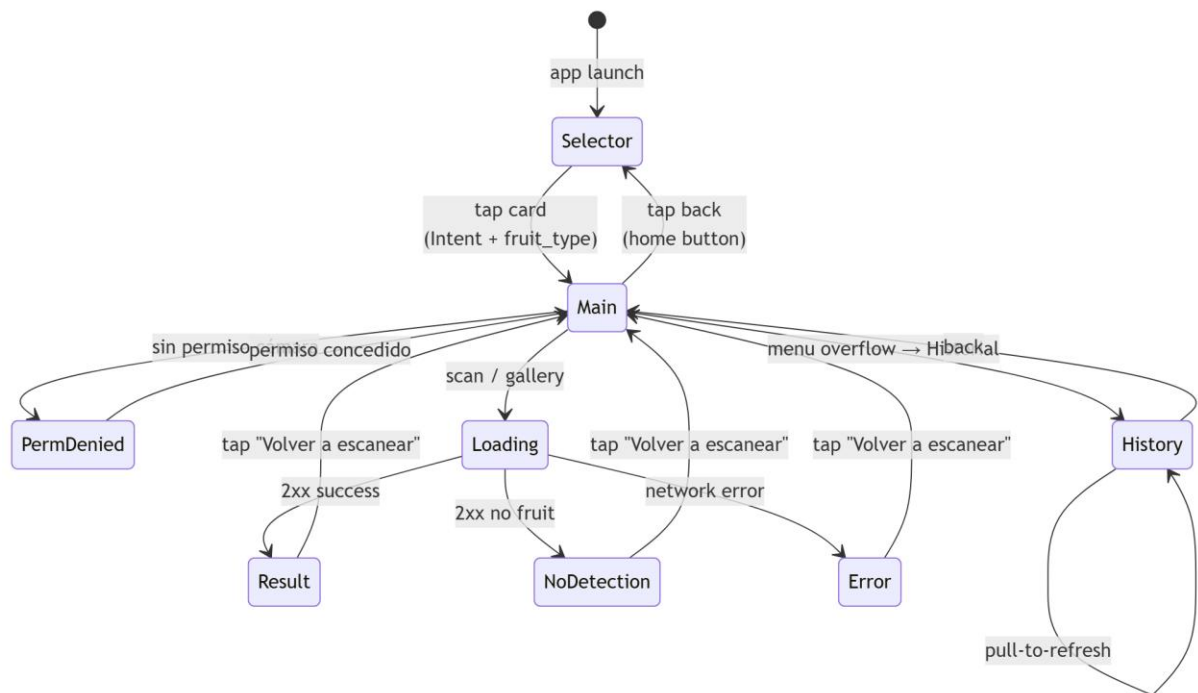


Figura 6. Máquina de estados de ScanState

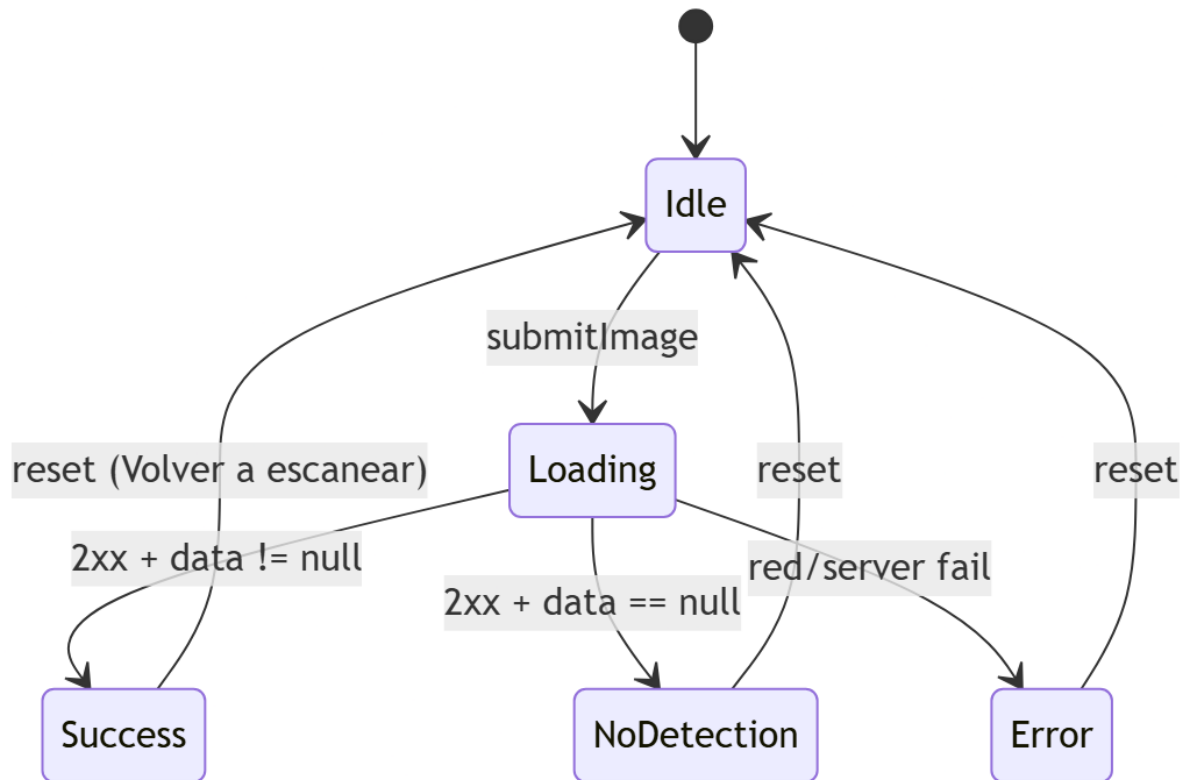


Figura 7. Transiciones válidas entre estados

3.3 Vista de Desarrollo

La Vista de Desarrollo describe la organización estática del código desde la perspectiva del programador.

Arquitectura backend — Layered / Clean:

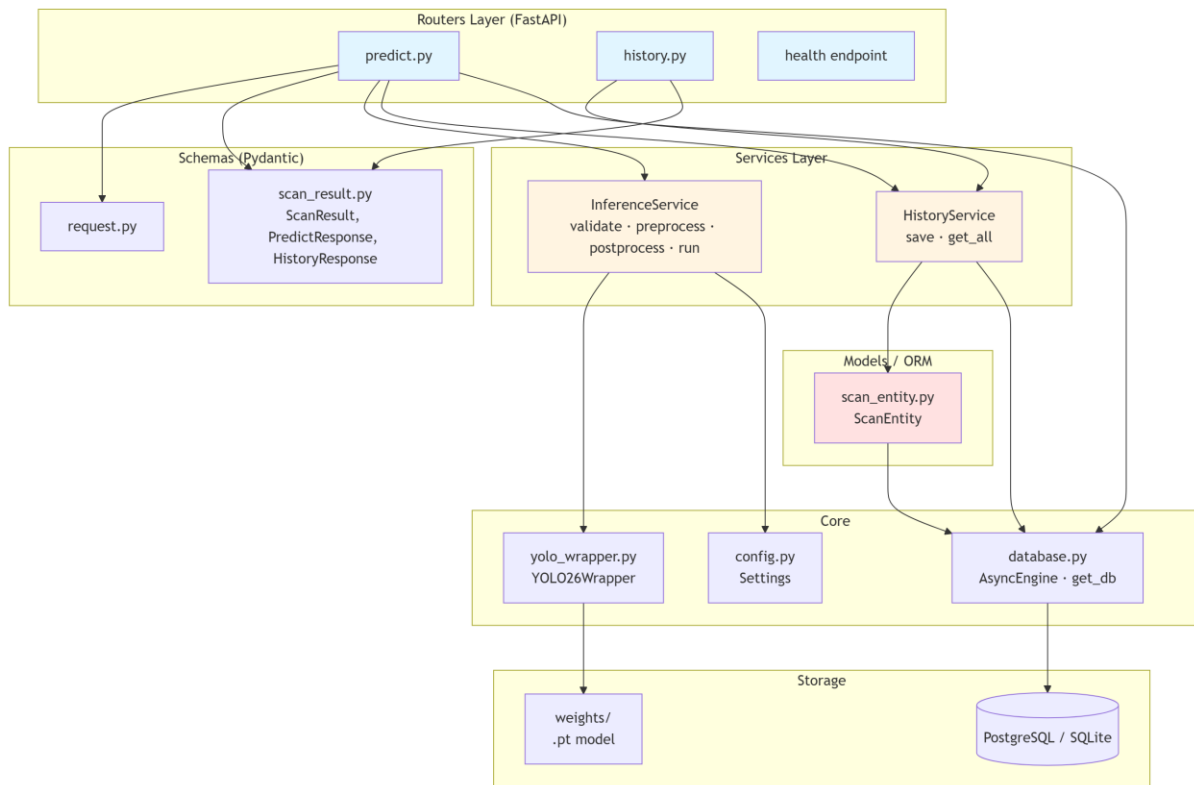


Figura 8. Arquitectura en capas del backend

- routers/ — Capa HTTP: routing, validación, manejo de respuestas.
- services/ — Lógica de negocio: InferenceService, HistoryService.
- core/ — Infraestructura: YOLO26Wrapper, config.py, database.py.
- models/ — ORM: ScanEntity (SQLAlchemy).
- schemas/ — DTOs: ScanResult, PredictResponse, HistoryResponse (Pydantic).

Principios aplicados:

- Dependencia direccional estricta: routers → services → models. Inversiones prohibidas.
- Inyección de sesión de BD por FastAPI Depends (override en tests).
- Lazy imports de ultralytics/torch para acelerar cold start.
- Settings única fuente de truth para configuración (pydantic-settings).

Arquitectura frontend — MVVM:

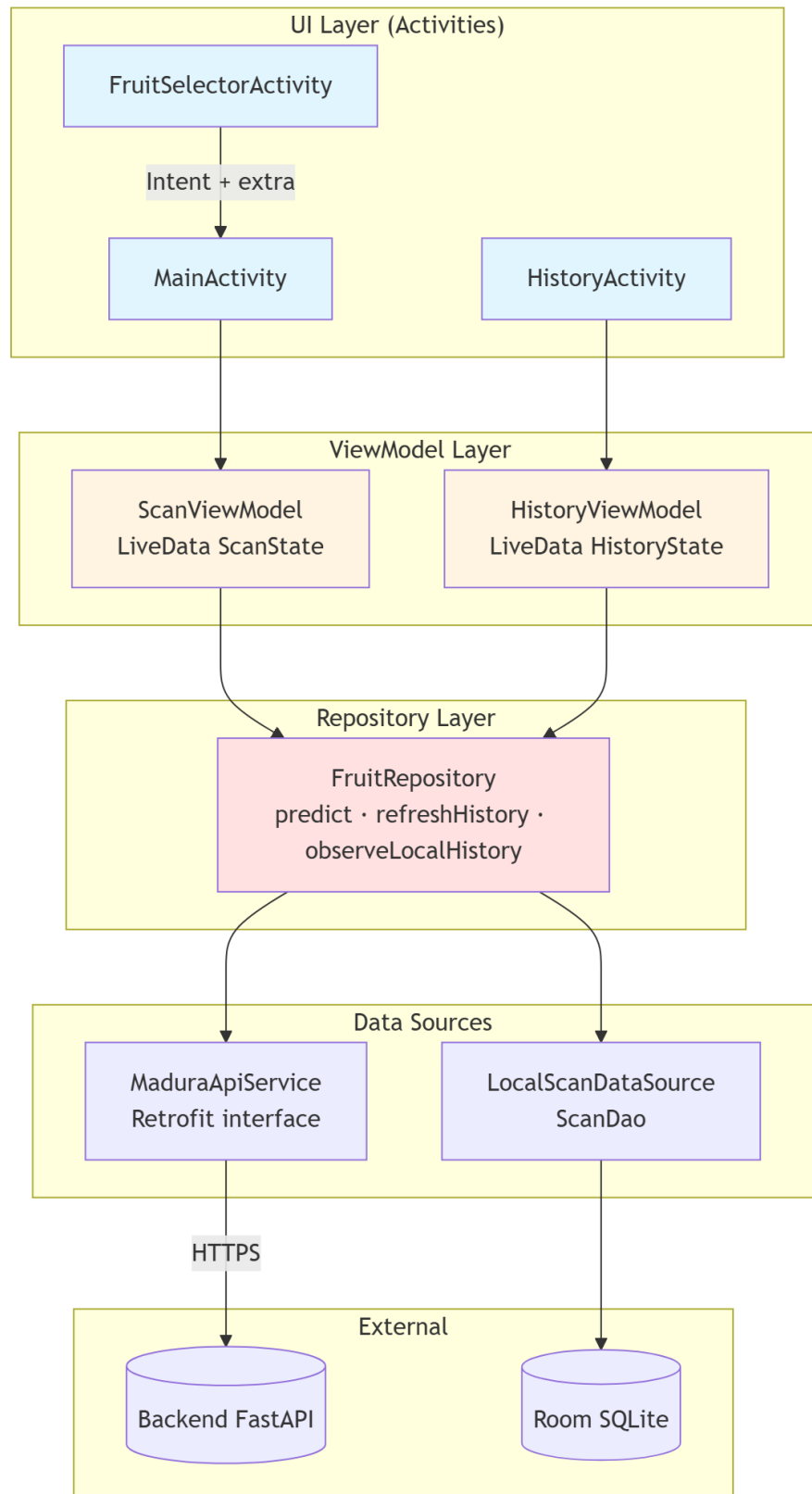


Figura 9. Arquitectura MVVM del frontend Android

- Activity / Layout — capa de presentación.
- ViewModel (`ScanViewModel`, `HistoryViewModel`) — estado UI, `LiveData`.
- Repository (`FruitRepository`) — Single Source of Truth: API + Room.

- Data Sources: MaduraApiService (Retrofit) + LocalScanDataSource (Room).

Convenciones de código:

Aspecto	Python (backend)	Kotlin (Android)
Estilo	PEP 8, 100 chars max	Kotlin Coding Conventions
Tipado	Type hints obligatorios	Kotlin null safety, val preferido
Documentación	Docstrings en services/schemas	KDoc en clases públicas
Commits	Conventional Commits (feat:, fix:, docs:, ...)	Idem
Líneas	Máx 100 chars	Máx 100 chars

CI/CD:

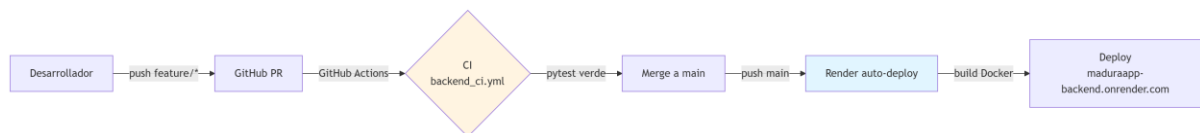


Figura 10. Flujo de integración y despliegue continuo (CI/CD)

- GitHub Actions ejecuta pytest en cada push/PR — sin merge a main con CI rojo.
- Render auto-deploy desde rama main en cada push exitoso.

3.4 Vista Física

La Vista Física documenta la topología de despliegue, los nodos hardware/software y los protocolos de comunicación.

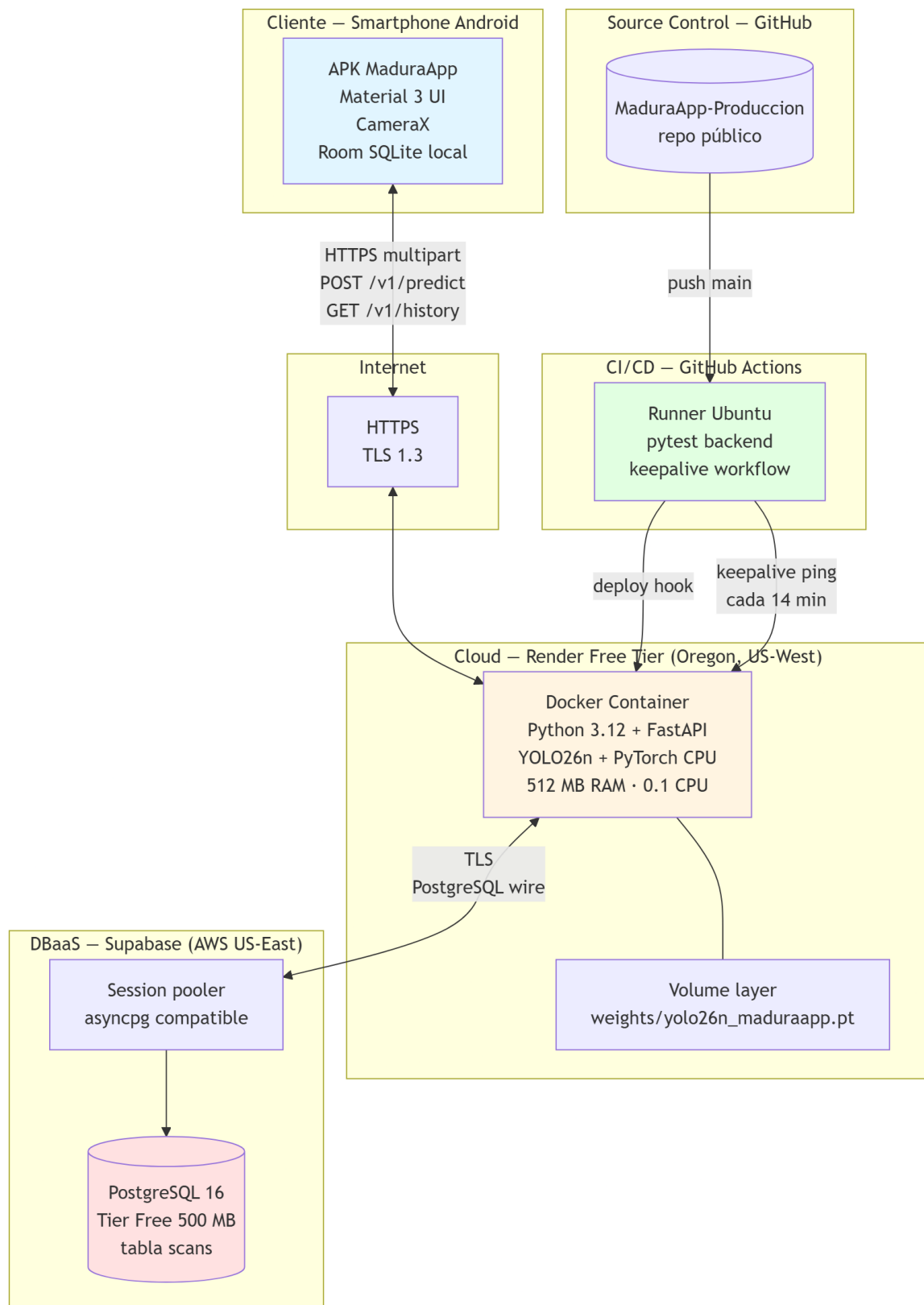


Figura 11. Topología general del sistema

Nodos del sistema:

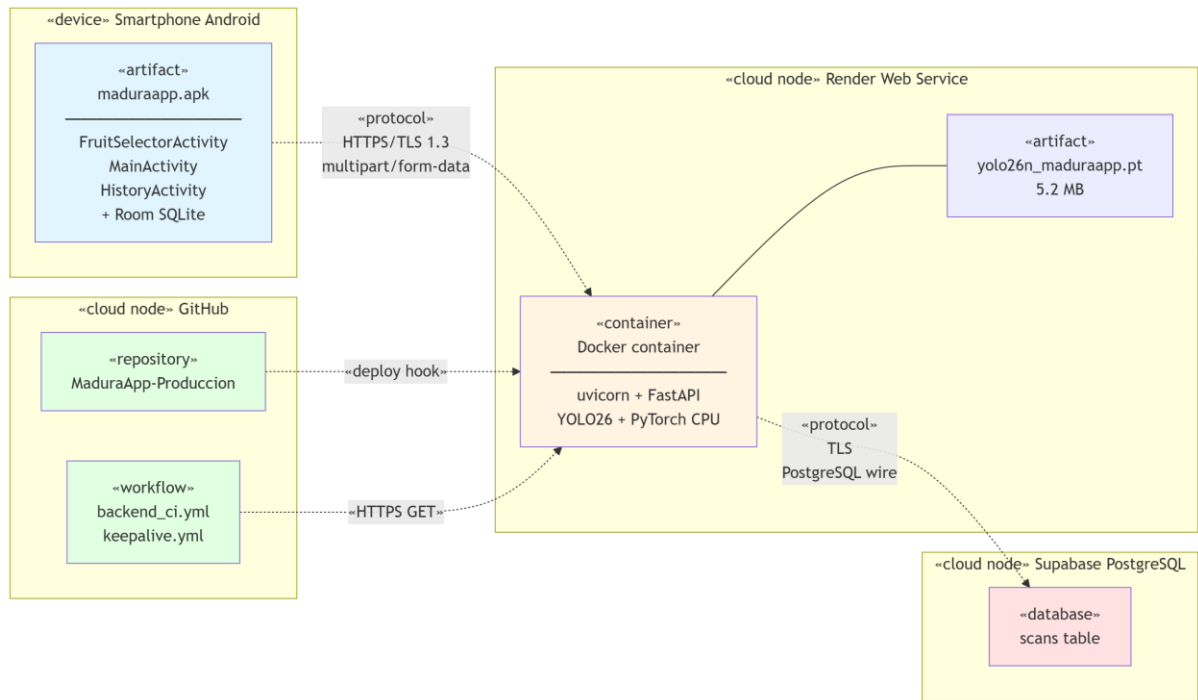


Figura 12. Diagrama de despliegue

Nodo	Plataforma	Especificaciones clave
Cliente Android	Smartphone físico / Emulador AVD	API 29+, 2 GB RAM mín, cámara autofocus
Backend	Render Free — Docker Linux Oregon	512 MB RAM, 0.1 vCPU, TLS incluido
Base de datos	Supabase PostgreSQL 16 (DBaaS)	Free tier 500 MB, AWS US-East
CI/CD	GitHub Actions Ubuntu runners	Ephemeral, pytest en cada commit

Protocolos de comunicación:

Enlace	Protocolo	Detalles
Android ↔ Render	HTTPS / TLS 1.3	multipart/form-data, timeout 30s
Render ↔ Supabase	TLS sobre PostgreSQL wire	asyncpg driver, SSL obligatorio
GitHub ↔ Render	HTTPS webhook	Auto-deploy en push a main
GitHub Actions ↔ Render	HTTPS GET /v1/health	Keepalive cada 14 min

Consideraciones de seguridad de red:

Riesgo	Mitigación implementada
Interceptación de tráfico	HTTPS obligatorio extremo-a-extremo
Exfiltración de credenciales	Variables sensibles en Render env, .env en .gitignore
DoS por imágenes gigantes	Validación tamaño 10 MB + Content-Type allowlist

Riesgo	Mitigación implementada
SQL injection	Acceso a BD exclusivamente vía ORM (SQLAlchemy parametrizado)
CORS abuse	Allowlist explícita en main.py (no *)

3.5 Vista de Escenarios (+1)

La Vista de Escenarios actúa como integradora de las otras cuatro, validando que el diseño funciona end-to-end para los casos de uso reales.

Casos de uso identificados:

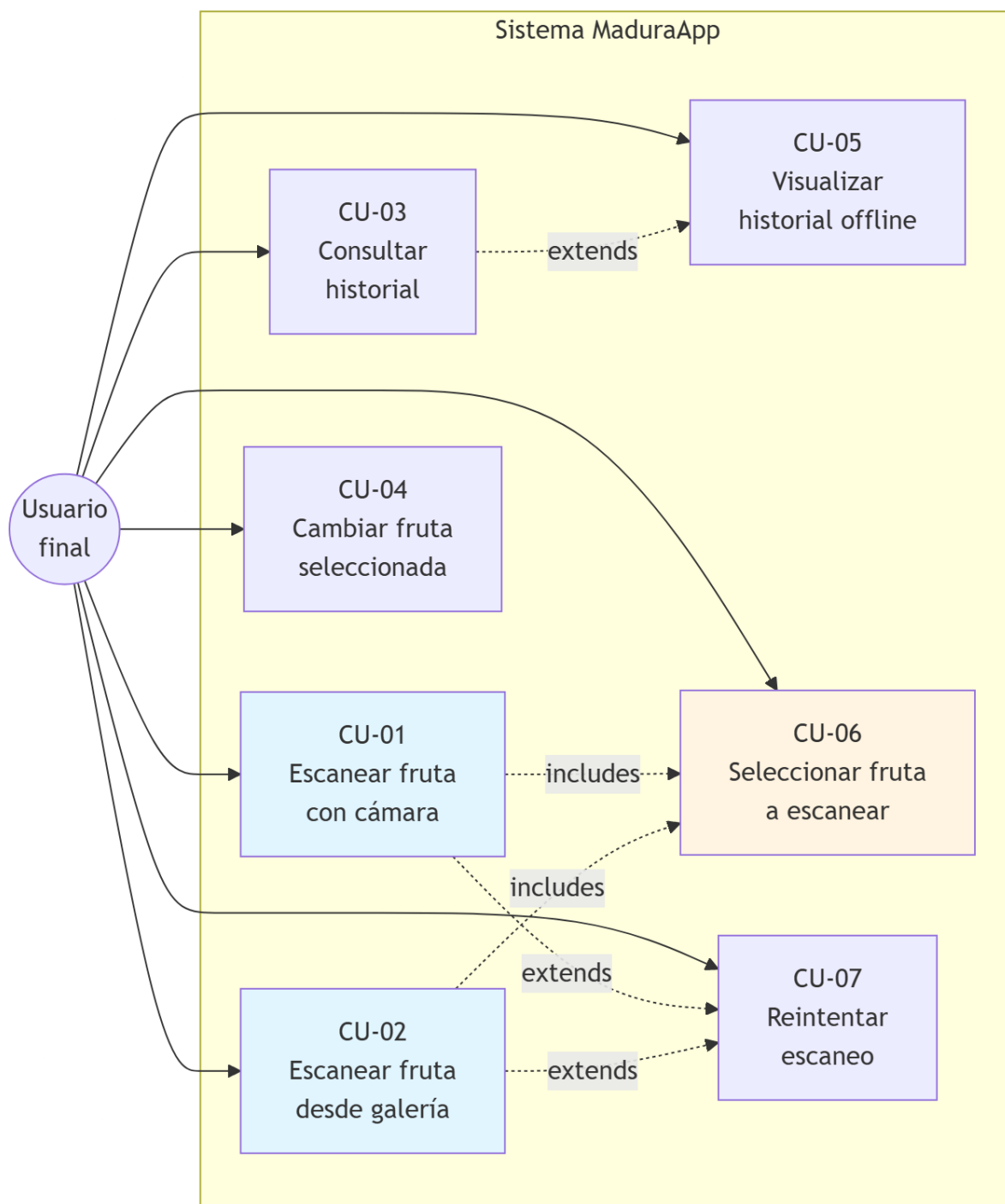


Figura 13. Diagrama de casos de uso

ID	Caso de uso	Descripción
CU-01	Escanear fruta con cámara	Flujo principal — captura, inferencia, resultado
CU-02	Escanear fruta desde galería	Fuente alternativa: foto pre-existente del dispositivo
CU-03	Consultar historial	Lista paginada de escaneos anteriores (remoto + cache local)
CU-04	Cambiar fruta seleccionada	Navegación inversa al selector de fruta
CU-05	Operar offline	Visualización de historial cacheado sin conexión

ID	Caso de uso	Descripción
CU-06	Seleccionar fruta a escanear	Pantalla inicial — grid 2×2 con las 4 frutas
CU-07	Reintentar escaneo	Reset del estado tras error o no detección

Trazabilidad CU-01 con las 4 vistas:

Vista	Elementos ejercitados en CU-01
Lógica	FruitSelectorActivity, ScanViewModel, FruitRepository, InferenceService, ScanResult, ScanEntity
Procesos	viewModelScope coroutine, async.to_thread, AsyncSession, ScanState: Idle→Loading→Success
Desarrollo	Módulos ui/, data/, routers/, services/, core/. Retrofit, OkHttp, asyncpg, Ultralytics
Física	Android → HTTPS → Render container → asyncpg → Supabase PostgreSQL

4. Mockups y Diseño de Interfaz

Los wireframes originales se encuentran en la documentación de la Evaluación 1. En el Sprint 2 se incorporaron tres pantallas nuevas o actualizadas:

Pantalla	Estado	Descripción
Selector de fruta inicial	Nueva (Sprint 2)	Grid 2×2 con cards: aguacate, plátano, tomate cherry, mango. Pantalla LAUNCHER.
Pantalla principal con fruta	Actualizada	Título dinámico con fruta seleccionada + botón de galería.
Resultado de escaneo	Actualizada	Semáforo de madurez + recomendación agronómica detallada.

4.1 Sistema de diseño

Aspecto	Decisión
Material Design	Material 3 (Material Components for Android 1.12)
Orientación	Portrait fija — simplifica layouts de cámara
Emojis de frutas	48sp — sustituyen imágenes vectoriales, reducen APK, universales
Colores semánticos	Verde (INMADURO), Amarillo (ÓPTIMO), Rojo (SOBRE_MADURO)

4.2 Pantalla 1 — Selector de fruta (nueva)

Toolbar verde con el nombre de la app, título de instrucción (22sp), subtítulo explicativo y grid 2×2 de cards (CardView 160dp, corner 16dp, elevation 3dp). Cada card muestra emoji grande + nombre de la fruta. El card de Tomate Cherry incluye un subtítulo adicional 'Variedad cherry o pera' (10sp, 65% alpha) porque el modelo fue entrenado con ese dataset específico.

Al tocar una card, lanza MainActivity con `EXTRA_FRUIT_TYPE` = identificador interno (ej. `aguacate_hass`).

4.3 Pantalla 2 — MainActivity actualizada

Cambios respecto a Evaluación 1:

Elemento	Evaluación 1	Evaluación 2 (actual)
Título toolbar	Estático: 'Análisis de madurez'	Dinámico: 'Escaneando {fruta}' si hay <code>fruit_type</code>
Botón galería	No existía	Nuevo: 🖼️ Seleccionar de galería
Back button	No existía	Regresa a FruitSelectorActivity

Elemento	Evaluación 1	Evaluación 2 (actual)
Botones de acción	Solo Escanear + Reintentar	Grupo actionButtons con Escanear + Galería

4.4 Flujo de navegación

App launch → FruitSelectorActivity → MainActivity (con fruta seleccionada) → (Escanear / Galería) → Loading → Resultado → (Reintentar / Historial / Volver).

La flecha de retroceso en MainActivity devuelve al selector. HistoryActivity es accesible desde el menu overflow de MainActivity.

5. Configuración del Ambiente de Pruebas

El ambiente de pruebas replica fielmente el ambiente de producción manteniendo paridad en versión de lenguaje, dependencias, esquema de base de datos y endpoints.

5.1 Matriz de paridad

Aspecto	Producción	Pruebas	¿Paridad?
Versión Python	3.12	3.12	✓ Sí
Framework web	FastAPI 0.135 + Uvicorn	FastAPI 0.135 + Uvicorn --reload	✓ Sí
Modelo ML	yolo26n_maduraapp.pt (5.2 MB)	Mismo archivo	✓ Sí
ORM	SQLAlchemy async 2.0	SQLAlchemy async 2.0	✓ Sí
Driver BD	asyncpg (PostgreSQL)	aiosqlite (SQLite)	⚠ Driver distinto, mismo dialecto
ENVIRONMENT	production	development	⚠ Controlado (warmup en dev)
TLS / HTTPS	Sí (Let's Encrypt)	No (HTTP local)	⚠ Configurado en cliente
CONFIDENCE_THRESHOLD	0.55 (env var)	0.55 (config.py default)	✓ Sí

5.2 Requisitos del entorno host

Software	Versión	Verificación
Python	3.12.x	python --version
Git	2.30+	git --version
Java	17 (LTS)	java -version
Android Studio	Hedgehog 2023.1+	Apertura sin errores
Docker (opcional)	24+	docker --version

5.3 Procedimiento de levantamiento del backend local

Paso 1 — Clonar el repositorio:

```
git clone https://github.com/apotheosiss/MaduraApp-Produccion.git
cd MaduraApp-Produccion/Producto/backend
```

Paso 2 — Crear y activar el entorno virtual:

```
python -m venv .venv
.venv\Scripts\Activate.ps1      # Windows PowerShell
source .venv/bin/activate      # macOS / Linux
```

```
pip install -r requirements.txt
```

Paso 3 — Configurar variables de entorno:

```
cp .env.example .env
# Editar .env — los valores por defecto son correctos para desarrollo local
```

Paso 4 — Aplicar migraciones y levantar el servidor:

```
alembic upgrade head
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Paso 5 — Verificar:

```
curl http://localhost:8000/v1/health
# Resultado esperado: {"status":"ok","model_loaded":true}
```

5.4 Configuración del cliente Android

Editar Producto/frontend/gradle.properties:

```
maduraapp.api.baseUrl=http://10.0.2.2:8000/ # emulador
maduraapp.api.baseUrl=http://localhost:8000/ # dispositivo físico + adb
reverse
```

Para demo con dispositivo físico + cable USB:

```
adb reverse tcp:8000 tcp:8000
```

5.5 Checklist de verificación del ambiente

#	Verificación	Comando / Acción
1	Python 3.12 disponible	python --version
2	Dependencias instaladas	python -c "import fastapi, ultralytics; print('OK')"
3	Modelo presente	ls Producto/backend/weights/yolo26n_maduraapp.pt
4	Backend arranca	uvicorn app.main:app → ver 'startup complete'
5	Health check OK	curl localhost:8000/v1/health → model_loaded:true
6	9/9 tests pasan	pytest tests/ -v
7	App Android compila	./gradlew assembleDebug
8	Smoke test E2E	Escanear fruta desde app → resultado visible

6. Configuración del Servidor de Producción

MaduraApp se despliega como servicio web containerizado en Render (PaaS), conectándose a PostgreSQL gestionado en Supabase. La configuración se gestiona como código (IaC) mediante `render.yaml` versionado en el repositorio.

6.1 Justificación de Render

Criterio	Render	Heroku	AWS App Runner	Railway
Free tier funcional	✓ 512 MB	✗ Eliminado 2022	✗ Requiere tarjeta	△ Trial limitado
Docker nativo	✓ Sí	△ Buildpacks	✓ Sí	✓ Sí
Auto-deploy Git	✓ Sí	✓ Sí	✓ Sí	✓ Sí
TLS auto	✓ Let's Encrypt	✓ Sí	✓ Sí	✓ Sí
Blueprint as code	✓ <code>render.yaml</code>	✓ <code>app.json</code>	△ CDK/CFN	△ Limitado

6.2 Variables de entorno en producción

Variable	Valor	Sensibilidad
ENVIRONMENT	production	Pública
DB_URL	postgresql+asyncpg://... (Supabase)	Secreta (sync:false)
AUTH_SECRET_KEY	Auto-generada por Render	Secreta (generateValue:true)
YOLO_MODEL_PATH	weights/yolo26n_maduraapp.pt	Pública
CONFIDENCE_THRESHOLD	0.55	Pública

6.3 Procedimiento de despliegue desde cero

- 1. Crear BD en Supabase → New Project → copiar connection string.
- 2. Cambiar prefijo: `postgresql://` → `postgresql+asyncpg://`
- 3. Aplicar migraciones localmente: `alembic upgrade head` (apuntando a Supabase).
- 4. En Render → New → Blueprint → conectar repositorio MaduraApp-Produccion.
- 5. Render detecta `render.yaml` automáticamente → apply.
- 6. Configurar `DB_URL` en Render Dashboard → Environment.
- 7. Verificar: `curl https://[URL-render]/v1/health` → `{status:ok, model_loaded:true}`
- 8. Actualizar `API_BASE_URL` en `gradle.properties` con la URL de Render.

6.4 Mitigación de cold start (Free tier)

Render Free suspende el contenedor tras 15 minutos de inactividad (cold start ~30-60 s). Se mitiga con:

- Workflow keepalive.yml en GitHub Actions: pinga /v1/health cada 14 minutos (cron */14 * * * *).
- Warmup deshabilitado en producción (ENVIRONMENT=production) para conservar RAM.
- gc.collect() + del de tensores después de cada inferencia.

7. Gestión de Datos — Backup y Replicación

Se documentan los procedimientos operativos para mantener copias de seguridad de la base de datos de producción y replicarla en el ambiente de pruebas.

7.1 Esquema de la base de datos

La BD de producción consta de una sola tabla (scans), creada por la migración Alembic 0001:

```
CREATE TABLE scans (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  timestamp   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  user_token  VARCHAR(255) NOT NULL DEFAULT 'anonymous',  
  fruit_type  VARCHAR(50)  NOT NULL,  
  maturity_label VARCHAR(50) NOT NULL,  
  confidence  REAL NOT NULL,  
  image_hash  VARCHAR(64)  
);
```

7.2 Procedimiento A — Backup completo de producción

Usando pg_dump v16:

```
export DB_URL_PROD="postgresql+asyncpg://user:pass@host:5432/postgres"  
  
# Convertir prefijo SQLAlchemy a formato libpq  
PG_URL="${DB_URL_PROD/postgresql+asyncpg/postgresql}"  
  
# Generar dump comprimido  
pg_dump "$PG_URL" --no-owner --no-acl --format=plain \  
  --table=scans --table=alembic_version \  
  | gzip > backup_prod_$(date +%Y-%m-%d_%H%M).sql.gz
```

Frecuencia recomendada: diario a las 03:00 + manual antes de cada migración.

7.3 Procedimiento B — Restaurar en ambiente de pruebas

```
# Crear BD local vacía  
createdb -h localhost -U postgres maduraapp_test  
  
# Restaurar el dump  
gunzip -c backup_prod_2026-05-26_2130.sql.gz | \  
  psql -h localhost -U postgres -d maduraapp_test  
  
# Verificar  
psql -h localhost -U postgres -d maduraapp_test \  
  -c 'SELECT COUNT(*) FROM scans;'
```

7.4 Replicación de configuración del servidor

El método más rápido aprovecha que render.yaml es portable: crear nuevo servicio en Render → Blueprint → conectar fork del repositorio → configurar DB_URL con nueva BD → deploy. El stack es idéntico en una URL distinta.

7.5 Política de retención

Tipo de backup	Retención	Almacenamiento
Backups manuales locales	30 días (rotación automática)	./dumps/ en máquina responsable
Backups pre-deploy	90 días	Repositorio privado cifrado
Autobackup Supabase	7 días (tier free)	Gestionado por Supabase
Snapshots evaluación académica	Permanente	Repo privado, cifrado AES-256

8. Plan de Pruebas

El plan de pruebas adopta el modelo de pirámide de pruebas y clasifica los tests en tres niveles según el indicador de logro IL2.2.

8.1 Distribución actual

Nivel	Cantidad	Framework	Frecuencia
Unitarios + integración	9 backend (pytest)	pytest + FastAPI TestClient	Cada commit (CI)
Unitarios JVM	~10 Android	MockK + JUnit + Turbine	Cada commit (CI)
Operacionales	8 manuales	curl + UI inspection	Pre-demo
E2E manual	8 pasos	App real + backend	1× por evaluación

8.2 Pruebas operacionales

ID	Test	Objetivo	Frecuencia
OP-01	Health check del backend	/v1/health retorna model_loaded:true	Continuo (keepalive)
OP-02	TLS válido	Certificado Let's Encrypt vigente	Mensual
OP-03	BD Supabase responde	Conexión exitosa via pooler	Diario (via health)
OP-04	App Android instala	APK debug en emulador y dispositivo	Cada release
OP-05	Smoke test cliente	curl a endpoint de producción	Pre-demo
OP-06	Cold start aceptable	Render despierta en < 60 s	Mensual
OP-07	Backups recuperables	Restaurar dump en BD efímera	Mensual
OP-08	Storage Supabase	< 450 MB (90% del free tier)	Semanal

8.3 Pruebas de validación

ID	Test	RF cubierto	Implementación
VAL-01	POST /v1/predict con imagen + fruit_type → ScanResult	RF-04, RF-05	test_predict_with_fruit_filter
VAL-02	Content-Type inválido → 400	RF-11	test_predict_invalid_content_type
VAL-03	Sin archivo → 422	RF-11	test_predict_no_file
VAL-04	fruit_type inválido → 400	RF-11	test_predict_invalid_fruit_type
VAL-05	Sin detección → success:false	RF-04	test_predict_no_detection
VAL-06	Historial vacío → lista vacía	RF-08	test_history_empty

ID	Test	RF cubierto	Implementación
VAL-07	Historial tras save → contiene registro	RF-07, RF-08	test_history_after_save
VAL-08	Historial paginado (limit/offset)	RF-08	test_history_pagination
VAL-09	Sin token → lista anonymous	RF-08	test_history_no_token
VAL-10	Submit imagen → Loading → Success	RF-04, RF-10	ScanViewModelTest
VAL-11	Cache local refleja response	RF-09	FruitRepositoryTest

8.4 Pruebas de verificación

ID	Test	KPI	Resultado actual
VER-01	Latencia inferencia sin TTA (local CPU)	< 200 ms p95	~80-150 ms ✓
VER-02	Latencia inferencia con TTA (local CPU)	< 400 ms p95	~160-300 ms ✓
VER-03	Latencia E2E Android → resultado (Render)	< 3 s p95	~1.5-2.5 s ✓
VER-04	mAP@50 del modelo en test set	≥ 0.75	0.9229 ✓
VER-05	Precision por clase	≥ 0.70 c/u	Todas > 0.85 ✓
VER-06	RAM en runtime durante inferencia	< 512 MB (free tier)	~400-490 MB ✓
VER-07	CI verde en cada commit a main	100% gates	✓ (vigente)
VER-08	Sin secretos en repo	0 hallazgos	✓ (verificado)

8.5 Criterios de aceptación del Sprint 2

Criterio	Estado
Suite pytest backend: 9/9 passing	✓ CUMPLIDO
Suite JVM Android: todos passing	✓ CUMPLIDO
CI verde en última versión de main	✓ CUMPLIDO
KPI mAP@50 ≥ 0.75 (actual: 0.9229)	✓ CUMPLIDO
Backend desplegado en Render y responde /v1/health	✓ CUMPLIDO
APK construible sin errores	✓ CUMPLIDO
Sin secretos en el repo público	✓ CUMPLIDO
Documentación 4+1 completa	✓ CUMPLIDO

9. Desarrollo de Software — Estado y Avances

9.1 Cobertura de requerimientos funcionales

RF	Descripción	Estado
RF-01	Capturar imagen con cámara	✓ Implementado — MainActivity.takePictureAndSubmit
RF-02	Seleccionar imagen desde galería	✓ Implementado — MainActivity.galleryLauncher (nuevo Sprint 2)
RF-03	Pre-seleccionar fruta	✓ Implementado — FruitSelectorActivity (nuevo Sprint 2)
RF-04	Enviar imagen al backend	✓ Implementado — FruitRepository.predict
RF-05	Clasificar madurez con IA	✓ Implementado — InferenceService + YOLO26Wrapper
RF-06	Recomendación contextualizada	✓ Implementado — RECOMMENDATION_MAP (12 combos)
RF-07	Persistir cada escaneo	✓ Implementado — HistoryService.save
RF-08	Historial paginado	✓ Implementado — GET /v1/history
RF-09	Historial offline	✓ Implementado — Room + LocalScanDataSource
RF-10	Semáforo visual	✓ Implementado — ColorCode + ripenessIndicator
RF-11	Validación de imagen	✓ Implementado — predict.py + validate_image
RF-12	Health check	✓ Implementado — GET /v1/health

Cobertura: 12/12 (100%) de requerimientos funcionales aprobados.

9.2 Avances desde la Evaluación 1

Avance	Impacto	Componentes afectados
Selector de fruta inicial	Filtra 12 clases → 3 (mejor precisión)	FruitSelectorActivity, ScanViewModel, predict.py
Selección desde galería	Analizar fotos pre-existentes	MainActivity, galleryLauncher, uriToJpegBytes
Test Time Augmentation (TTA)	+2-3% precisión con filtro activo	yolo_wrapper.py (augment), inference_service.py

Avance	Impacto	Componentes afectados
Umbral ajustado 0.45 → 0.55	Menos falsos positivos	config.py, render.yaml
Recomendaciones agronómicas	Información útil y contextual al usuario	RECOMMENDATION_MAP (12 entradas)
Optimización RAM producción	Free tier (512 MB) funcional	yolo_wrapper.py (warmup condicional), gc.collect

9.3 Métricas del modelo IA

Métrica	Resultado	KPI	Estado
Arquitectura	YOLO26n (Ultralytics Jan 2026)	—	—
Épocas	80	—	—
Dataset	31.940 imágenes, 12 clases, 4 frutas	—	—
Tamaño del modelo	5.2 MB	< 10 MB	✓
mAP@50	0.9229	≥ 0.75	✓ +17.3 pts
Precision avg	0.945	≥ 0.70	✓
Recall avg	0.873	≥ 0.65	✓
Inferencia CPU local	~80-150 ms	< 200 ms	✓
Hardware entrenamiento	Kaggle Tesla T4 ×2	—	—

9.4 Seguridad del software

Vector	Mitigación implementada
SQL injection	ORM-only access, sin string formatting en queries
DoS por imágenes gigantes	Límite 10 MB + Content-Type allowlist
MITM	HTTPS obligatorio en producción (TLS 1.2+ por Render)
Credenciales en repo	DB_URL y AUTH_SECRET_KEY solo en Render env, .env en .gitignore
CORS abuse	Allowlist explícita en main.py
Cleartext Android	usesCleartextTraffic=true solo en debug; release requiere HTTPS

10. Conclusiones y Lecciones Aprendidas

10.1 Cumplimiento de objetivos

Al cierre de esta Evaluación 2, el proyecto MaduraApp cumple con:

Indicador	Estado	Evidencia
IL2.1 — Diseño metodológico con diagramas	✓ CUMPLIDO	5 vistas Kruchten + mockups + ERS + diagramas UML
IL2.2 — Ambiente de pruebas configurado	✓ CUMPLIDO	DEMO_INICIO.ps1 + checklist 8 pasos + pytest 9/9
IL2.3 — Producto funcional de calidad	✓ CUMPLIDO	12/12 RF implementados + CI verde + mAP@50 = 0.9229

10.2 Decisiones técnicas relevantes

Decisión	Razonamiento
Render Free vs Standard	Costo cero para proyecto académico; restricciones de RAM mitigadas con warmup condicional y gc.collect
Supabase vs Neon.tech	Mejor integración con el flujo del equipo y pooler de conexiones nativo
YOLO26n vs YOLOv8/v11	Mejor compromiso tamaño/precisión: 5.2 MB con mAP@50 = 0.92
fruit_type opcional	UX más flexible: modo libre (12 clases) + modo filtrado (3 clases); el backend degrada limpio
TTA solo con filtro	2× latencia solo cuando vale la pena (ya sabemos la fruta)
Dos repositorios GitHub	Separación código de testeo (MaduraApp) vs proyecto académico formal (MaduraApp-Produccion)

10.3 Lecciones aprendidas

- La gestión de RAM en cloud free tiers es crítica para modelos ML. Medir antes de desplegar.
- La paridad ambiente prueba/producción es un trabajo continuo, no un check único.
- El modelo 4+1 de Kruchten desde el diseño temprano evita decisiones arquitectónicas implícitas difíciles de revertir.
- Conventional Commits + Git flow aceleran la trazabilidad y la generación de notas de release.
- El filtrado por tipo de fruta antes de inferencia mejora la experiencia del usuario de forma significativa.

10.4 Trabajo futuro

- Implementación de autenticación real (Bearer JWT firmado).
- Migración a Render Standard si el proyecto se mantiene activo post-asignatura.

- Expansión del catálogo de frutas soportadas (kiwi, papaya, palta Edranol).
- Modo offline-first: modelo TFLite embebido en el APK.
- Tests E2E automatizados con Espresso para RF-01, RF-02 y RF-03.

11. Anexos

A.1 Endpoints REST

Método	Path	Request	Auth
POST	/v1/predict	multipart: file (JPEG/PNG) + fruit_type (opcional)	Bearer opcional
GET	/v1/history	?limit=N&offset=M	Bearer opcional
GET	/v1/health	—	No requerida

A.2 Comandos frecuentes

Backend:

```
cd Producto/backend && source .venv/bin/activate
uvicorn app.main:app --reload --port 8000
pytest tests/ -v
alembic upgrade head
```

Android:

```
cd Producto/frontend
./gradlew assembleDebug
./gradlew test
adb reverse tcp:8000 tcp:8000 # demo con teléfono físico
```

Backup BD:

```
export DB_URL_PROD="postgresql+asyncpg://..."
./scripts/backup_prod.sh
```

A.3 Variables de entorno

Variable	Desarrollo	Producción
ENVIRONMENT	development	production
DB_URL	sqlite+aiosqlite:///./maduraapp_dev.db	postgresql+asyncpg://... (Supabase)
CONFIDENCE_THRESHOLD	0.55	0.55
YOLO_MODEL_PATH	weights/yolo26n_maduraapp.pt	weights/yolo26n_maduraapp.pt
AUTH_SECRET_KEY	dev_secret_key	Auto-generado por Render

A.4 Referencias

- Kruchten, P. (1995). Architectural Blueprints — The 4+1 View Model. IEEE Software, 12(6), 42-50.
- Chapman et al. (2000). CRISP-DM 1.0: Step-by-step data mining guide. SPSS Inc.
- FastAPI documentation: <https://fastapi.tiangolo.com>
- Ultralytics YOLO: <https://docs.ultralytics.com>
- Android CameraX: <https://developer.android.com/training/camerax>

- Render Blueprint Spec: <https://render.com/docs/blueprint-spec>
- FAO (2019). The State of Food and Agriculture 2019: Food loss and waste reduction. Rome.
- ODEPA: Informes técnicos post-cosecha frutas climatéricas, Chile 2023-2025.